Smart Serial Controller v2 (ZedBoard)

A software-defined, autonomous serial controller for the ZedBoard (Zynq-7020), in Verilog-2001. It is the v2 architecture of the slides:

- **Serial engine:** protocols run as short programs on two small state machines (8 instructions, with open-drain and CRC instructions for serial buses). UART, SPI, I2C, Manchester and the OLED driver are programs.
- **Pin crossbar:** any engine on any connector, switched while running; it never cuts a frame and measures every switch.
- **Network-on-chip:** a 2 x 3 mesh of routers (XY routing, wormhole, credits, priority) connects the engines, so protocol conversion is just a packet from one node to another.
- **Sensor hub + DMA writer:** the hardware reads the sensors on a schedule, makes 16-byte time-stamped records and writes them into DDR through S_AXI_HP0. The CPU wakes once per 32 readings.
- The **v1 UART, SPI and I2C engines** are kept, and v1 software still works (AXI-APB is used only for set-up).

```
ARM --AXI4-Lite--> APB --> registers (set-up only)
                                                    DMA writer --AXI4--> DDR (HP0)
 (0,0) serial engine -- (1,0) sensor hub -- (2,0) DMA writer
        |                      |                      |
 (0,1) UART ----------- (1,1) SPI ------------ (2,1) I2C       2 x 3 NoC
        \___________ pin crossbar: JA JB JC JD OLED LEDs ________/
```

Start here: **GETTING_STARTED.md** (install, simulate, build, run, step by step).

What is in the box

Verilog	`rtl/` (v1 engines, NoC, serial engine, crossbar, hub, DMA writer, core, AXI wrapper), `boards/zedboard/` (pads, two top levels, pin constraints)
Way 1: FPGA only	`zed2_top_standalone`: a boot sequencer sets everything up; PC frames, sensor records into block RAM, switches drive the crossbar
Way 2: ARM + FPGA	`zed2_top_ps` + block design (PS, controller, HP0) + the C console app in `sw/`
Serial-engine programs	`programs/*.se`, assembler `tools/se_asm.py`
Tests	13 self-checking unit testbenches + 2 system testbenches (`sim/`), all passing
NoC study	`noc_study/`: bus vs crossbar vs mesh at 4, 8, 16 nodes
Scripts	`vivado/*.tcl`, double-click `windows/*.bat`, Icarus `sim/run_*.sh`
Documentation	`docs/` (below), PDFs in `docs/pdf/`

Results so far (simulation)

claim (slides)	result	where
flash ID through the NoC (slide 17)	PC sends `01 9F 03`, gets `20 BA 19`	`tb_ssc2_system` [2], `tb_zed2_standalone` [2]
hub + DMA, CPU sleeps (slides 18-20)	32 records in DDR, correct fields and addresses; **0 register accesses and 1 interrupt per 32 readings** (v1 way: 10 accesses + 2 interrupts per reading)	`tb_ssc2_system` [3], [7]
protocols as programs (slides 12-13, 23)	UART, SPI and I2C programs match the v1 engines bit for bit; 115200 baud measured as 8680.0 ns per bit; Manchester with CRC works	`tb_se_programs` (69 checks)
runtime pin switching (slide 14)	never mid-frame; 2 clocks (20 ns) from an idle engine	`tb_xbar` (1.2 M checks), `tb_ssc2_system` [5]
NoC correctness (slide 23)	every packet once, unchanged, in order; no deadlock; 2 clocks per hop, 1 flit/clock streaming	`tb_noc_router`, `tb_noc_mesh`
area (slide 22)	about 18.5k LUTs, 7.7k FFs (yosys estimate) = 35 % of the Zynq-7020	`docs/MEASUREMENTS.md`
NoC scalability (slide 24)	the mesh loses on area and latency at 4-16 nodes; it matches the crossbar's throughput and its logic depth stays flat, so it gains only on clock speed at 16 nodes	`noc_study/README.md`

The board measurements (baud rate, SCL frequency, switch time, CPU load, Vivado area and Fmax) are for you to take with the procedures in docs/MEASUREMENTS.md.

Documentation

file	what
GETTING_STARTED.md	step-by-step: tools, simulation, Way 1, Way 2, troubleshooting
docs/CODE_WALKTHROUGH.md	every module explained, plus two end-to-end traces
docs/VERIFICATION.md	how to verify, and what correct output looks like (simulation and board)
docs/MEASUREMENTS.md	the six measurements of slide 24
docs/REGISTER_MAP.md	every register
docs/SPEC.md	the design contract (packet format, ports, behaviour)
`docs/blocks/*.md`	one detailed explanation per block: `noc`, `xbar`, `serial_engine`, `se_programs`, `ni_se`, `ni_uart`, `ni_spi_i2c`, `hub`, `dma_writer`
noc_study/README.md	the NoC scalability study

Quick commands

```
# simulation (WSL / Linux, Icarus Verilog)
sh sim/run_unit.sh                 # 13 unit tests, ~3 min
sh sim/run_system.sh               # 2 system tests, ~10 min
sh tools/build_programs.sh         # after editing programs/*.se

# Vivado (any OS), from this folder
vivado -mode batch -source vivado/run_sim.tcl -tclargs tb_ssc2_system
vivado -mode batch -source vivado/build_standalone.tcl     # Way 1 bitstream
vivado -mode batch -source vivado/build_ps_system.tcl      # Way 2 bitstream + XSA

# PC side of the slide-17 flow
python tools/pc_frame.py COM5 01 9F 03
```

Windows: `windows\1_simulate.bat`, `2_build_way1_fpga_only.bat`, `3_build_way2_arm_fpga.bat`, `4_open_project_gui.bat`, `5_noc_study_vivado.bat`.

Hardware

ZedBoard (Rev C/D), PmodUSBUART on JA (top row), PmodSF3 on JB, PmodTMP2 on JC pins 3-6/9-12, optional logic analyser on JD, the on-board OLED, LEDs and switches. Pin details: `boards/zedboard/zed2_pins.xdc` and GETTING_STARTED.md step 4b.